

# Introducción a la ingeniería de software. ¿Qué es?

## Software

El software es un conjunto de programas en conjunto con toda la documentación requerida para definir y mantener los programas ejecutables que se entregan al cliente

## Ingeniería de Software

Es una disciplina que se encarga desde la especificación del sistema hasta el mantenimiento del sistema

Esta ingeniería es importante ya que:

- se requiere producir de manera rápida y económica sistemas confiables.
- Resulta más barato a largo plazo utilizar métodos y técnicas de ingeniería de software

Para la ingeniería de software :

- se debe entender el problema y las necesidades del cliente antes de proponer una solución.
- El diseño es una actividad crucial en el desarrollo de un software para obtener calidad de software y facilidad de mantenimiento.

La ingeniería de software nace tras las crisis del software.

## Estado actual y antecedentes. La crisis del software.

Tiene que ver con que en la conferencia de OTAN se recalco la dificultad para generar software libre de defectos, fácilmente comprensibles y que sean verificables. Sin embargo, este término fue utilizado anteriormente por Dijkstra.

Causas

- La evolución de la tecnología del hardware permitió la posibilidad del desarrollo de sistemas más grandes y complejos. El salto en el hardware no fue acompañado por un salto en el desarrollo del software.
- En combinación con lo anterior, la demanda creciente de estos sistemas complejos, la subestimación de la complejidad que supone el desarrollo de software fueron las causas de esta crisis.

Muchos de los sistemas desarrollados fracasan debido a fallas en el software, que principalmente son consecuencia de dos factores:

- Demandas crecientes: la demanda de sistemas más grandes y complejos aumenta. Y Los usuarios necesitan que se construyan y distribuyan de manera rápida. La problemática aquí es que los métodos tradicionales de ingeniería no permiten lograr esta "espontaneidad".
- Bajas expectativas: conformidad de las empresas y de los usuarios ante un software, es decir, las empresas se conforman con que el software solo "funcione",

## Disciplinas que conforman la ingeniería de software.

- Disciplinas técnicas: nos referimos a actividades que aportan al desarrollo de software como producto. Ellas son:
  - Toma de requerimientos, Análisis de requerimiento, Diseño de software, Implementación de software, Prueba de software, Despliegue de un software, Documentación de software, Capacitaciones a usuarios.
- Disciplinas de gestión: hacen referencia a actividades de planificación, monitoreo y control del proyecto de desarrollo de software.
- Disciplinas de soporte: son disciplinas que permiten verificar la integridad y calidad de un producto de software. Entre ellas se incluyen:
  - Gestión de Configuración del Software (SCM)
  - Toma de métricas
  - Aseguramiento de calidad (QA - Quality Assurance)



## Ejemplos de grandes proyectos de software fallidos y exitosos.

### Proyectos fallidos

#### Ubuntu Phone

Hablando de Ubuntu Touch, el teléfono Ubuntu fue un fracaso. La interfaz de Ubuntu Touch era terrible.

#### Hannah Montana Linux

El objetivo de Hannah Montana Linux (HML) era atraer a los usuarios más jóvenes.

¿Qué era Hannah Montana Linux? Una distribución de Linux de color púrpura y rosa, con KDE como escritorio y el ambiente de canal de Disney al frente y al centro. Habría sido mejor si se hubieran añadido al menos algunas aplicaciones que resultaran atractivas para los usuarios, en lugar de hacer nada más que darle un tema a KDE.

### Proyectos exitosos

#### GNU y Linux:

GNU/Linux es el nombre completo del sistema operativo que normalmente todos conocemos como Linux.

#### Mozilla Firefox

Durante los años de dominio absoluto de Internet Explorer la Web estaba limitada pero todo comenzó a cambiar cuando llegó Firefox.

Firefox pronto fue más rápido, más completo, más seguro, más preciso con los estándares web y, por así decirlo, mejor en todo.

Ciclo de vida del proyecto y del producto (Abajo)

## Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software

Se denomina ciclo de vida a una serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo. Se define elementos del proceso y el flujo del proceso. Es decir, son modelos genéricos de los procesos de software.

Los modelos de proceso pueden incluir las mismas actividades, pero cada uno pondrá más énfasis en algunas respecto a otras y definirán flujos de procesos diferentes.

Los modelos de ciclos de vida especifican:

- Las fases del proceso (requerimientos, especificaciones, diseño, etc.)
- Orden en el cual se llevan a cabo.

### Relación entre el ciclo de vida del proyecto y del producto

No existe un impacto entre los ciclos de vida de cada uno, sino que el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar

→ esto va?

## Clasificación de los ciclos de vida

→ clasif. de modelos

### Modelo Secuencial

En Este modelo el proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada (lineal).

Se suele utilizar en aquellos sistemas donde los requerimientos se comprenden bien ya que éstos se congelan al finalizar la etapa de toma de requerimientos. Generalmente, este tipo de modelos está dirigido por documentos.

### Escenarios de proyectos donde elegirlo:

Sistemas que no requieren interacción continua con el cliente

Sistemas con requisitos bien definidos y estables

Proyectos de corta duración y bajo riesgo

Proyectos donde la documentación es obligatoria

### Dificultades

- Los proyectos raramente siguen un flujo secuencial.
- El modelo secuencial exige que los requerimientos sean completamente explícitos desde un principio y esto no suele suceder.
- Encontrar un defecto implica un gran rediseño en la solución
- Un equipo no puede comenzar hasta que el otro no haya finalizado.

### **Modelo iterativo/incremental**

Este modelo aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades. Cada iteración produce un incremento de software funcional potencialmente entregable. El sistema se desarrolla como una serie de versiones (incrementos) y cada una añade funcionalidades a la versión anterior. Este ciclo de vida se utiliza en metodologías ágiles.

### Ventajas frente al modelo secuencial

- el retrabajo es menor ya que en cada incremento se obtiene una retroalimentación que permite adaptar el modelo de las necesidades del cliente.
- El cliente es parte del proceso de desarrollo. En el modelo secuencial, el cliente debía esperar hasta la última instancia para recibir el producto y no era capaz de juzgar el avance.
- Muy útil para sistemas de requerimientos cambiantes.

### Dificultades desde una perspectiva administrativa

- Alto costo de documentar cada incremento.
- La estructura del software tiende a degradarse frente a una cantidad considerable de incrementos.

### **Modelo Recursivo**

El modelo recursivo es utilizado para gestionar los riesgos del desarrollo de sistemas complejos a gran escala. Requiere de la intervención del cliente, consta de 4 etapas recursivas:

1. Determinar objetivos.
2. Identificar y resolver los riesgos.
3. Desarrollar y probar.
4. Planificar la próxima iteración.

Para llevarlo adelante, se subdivide el proyecto en varios mini proyectos, intentando resolver en cada uno los riesgos más relevantes hasta que no quede ninguno.

### Dificultades desde una perspectiva administrativa

- Puede ser más costoso en tiempo y dinero
- La tecnología puede ser obsoleta.
- Puede carecer de mantenimiento o documentación, lo cual dificulta mucho las tareas.
- Se genera una versión del producto recién al final.



## Administración de proyectos dependiendo en ciclo de vida

La elección de un ciclo de vida afecta de forma directa a la administración que se debe realizar sobre el proyecto, ya que cada uno de los modelos de proceso poseen características y enfoques distintos.

Además, durante el desarrollo del proyecto, la gestión de este se realiza en consecuencia de esta elección, ya que no es lo mismo gestionar un proyecto que se desarrolla de manera iterativa, que uno que se desarrolla de manera secuencial o en cascada.

### Procesos de Desarrollo Empíricos vs. Definidos

forma en que se gestionan los procesos de desarrollo de software.

Un proceso de software se define como un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

### **Procesos Definidos**

Los procesos definidos son considerados deterministas: ante la misma entrada se pretende que se obtendrá la misma salida. Asumen que, si se aplican una y otra vez, se obtendrán siempre los mismos resultados, a pesar de cambiar de equipo.

1. Están inspirados en las líneas de producción.
2. La administración y el control provienen de la predictibilidad del proceso.
3. Puedo usar cualquiera de los ciclos de vida.

Son procesos estructurados, documentados y predecibles, donde cada fase, actividad y resultado esperado está claramente especificado.

### Procesos Empíricos

Los procesos empíricos NO es considerado determinista ya que no se asume que ante la aplicación del mismo proceso se obtendrán los mismos resultados. En estos procesos se pueden obtener diferentes resultados dependiendo del contexto en el cual se apliquen.

1. Se basan en ciclos cortos de inspección y adaptación (retroalimentación)
2. La administración y el control es por medio de inspecciones frecuentes
3. No se pueden combinar con cualquiera de los tipos de ciclos de vida, se suele recomendar el modelo iterativo-incremental

La experiencia que se obtiene solo funciona en ese mismo proyecto

Los procesos empíricos poseen un tercer pilar muy importante, que es la transparencia de la información ante todos los integrantes del equipo de trabajo.

### Criterios para la selección de un modelo de proceso

Es necesario evaluar los niveles de riesgo, la claridad y cantidad de requerimientos, los conocimientos del equipo de desarrollo, los plazos y la posibilidad de poder validar los resultados del trabajo, también el trabajo en equipo.

1. Si los requerimientos son claros y exhaustivos y el cliente no requiere obtener el producto rápidamente, es posible aplicar el modelo secuencial.
2. Si el cliente desea obtener resultados lo antes posible, con las funciones principales del sistema, entonces sería conveniente la elección del modelo iterativo- incremental.
3. Si el objetivo es disminuir los riesgos presentes en el desarrollo de sistemas grandes y complejos, entonces el modelo recursivo puede ser una buena opción.

### Características del producto

| Ciclo de Vida             | Procesos de desarrollo que lo admite | Características que lo hacen elegible                                                                                                                                                                                                                                                               |
|---------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Secuencial                | Definidos                            | <ul style="list-style-type: none"><li>• Alta certeza (baja incertidumbre).</li><li>• La organización tiene una forma tradicional de trabajar.</li><li>• No se pueden realizar entregas parciales del software (se debe entregar todo junto).</li><li>• Eliminar riesgos de planificación.</li></ul> |
| Iterativo/incr<br>emental | Definidos o Empíricos                | <ul style="list-style-type: none"><li>• Existe incertidumbre.</li><li>• Volatilidad de requerimientos.</li><li>• Posibilidad de entregas parciales.</li><li>• Uso en etapas tempranas del producto.</li></ul>                                                                                       |
| Recursivo                 | Definidos                            | <ul style="list-style-type: none"><li>• Mucho control de riesgos en cada iteración.</li><li>• No se pueden realizar entregas parciales.</li></ul>                                                                                                                                                   |

### Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software.

#### RELACIÓN ENTRE CICLO DE VIDA Y PROCESO DE DESARROLLO

El ciclo de vida define cómo se organiza el desarrollo

Los procesos de desarrollo son las actividades dentro de ese ciclo (ej: análisis, diseño, implementación, pruebas).

## Ciclo de vida de un proyecto

Ciclo de vida de un proyecto Software es una representación de un proceso, grafica una descripción del proceso

son Secuencial, Iterativo y Recursivo



## Componentes de un Proyecto de Sistemas de Información.

### QUE ES UN PROYECTO

Un proyecto de software es un esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único.

Define el proceso y tareas que se van a realizar, el personal que se encargará de las actividades y los mecanismos que se implementarán para valorar riesgos, controlar el cambio y evaluar la calidad.

### CARACTERÍSTICAS

**Únicos:** El resultado de un proyecto será único e irrepetible para cualquier proyecto, como por ejemplo el calendario o presupuestos.

**Orientado a objetivos:** los objetivos deben ser:

**No ambiguos:** deben ser lo suficientemente claros para guiar el desarrollo del proyecto.

**Medibles:** es necesario medir el objetivo para determinar el avance sobre el proyecto.

**Alcanzable:** deben ser factibles de realizarse por el equipo

**Duración limitada de tiempo:** Esto implica que un proyecto tiene un principio y un fin, liberando los recursos y personas. Cuando se alcanzan los objetivos del proyecto, este llega a su fin.

**Conjunto de tareas interrelacionadas:** Se definen las tareas, sus dependencias, los recursos y personas asignadas

### ADMINISTRACIÓN DE PROYECTO DE SOFTWARE

La administración de proyectos es la organización y coordinación de personas y recursos que permiten desarrollar un producto o servicio de acuerdo con el presupuesto, tiempo y funcionalidades acordadas con el cliente, asegurando que se entregue un producto o servicio de alta calidad. La administración de proyectos es una parte fundamental para que el proyecto sea exitoso. Esta disciplina tiene dos grandes actividades:

- Planificación.
- Monitoreo y control.

Metas más importantes de la administración de proyectos:

1. Entregar software al cliente en el tiempo acordado.
2. Mantener los costos dentro del presupuesto general.
3. Entregar software que cumpla con las expectativas del cliente.
4. Mantener un equipo de desarrollo óptimo y con buen funcionamiento.

Entonces la administración de proyecto Es la disciplina que organiza y controla personas + recursos para que el producto cumpla con:

- Tiempo (se entrega cuando corresponde).
- Costo (se respeta el presupuesto).
- Calidad / Alcance (funciona como el cliente lo espera).

Aquí entra la famosa triple restricción: Tiempo 🕒 Costo 💰 Alcance

### La triple restricción

Las variables de restricción de un proyecto son 3:

tiempo, costo y alcance. El balance de estos tres factores afecta directamente la calidad del proyecto. Es responsabilidad del Líder de proyecto balancear estas variables.

Alcance – ¿Qué trata de alcanzar?

Son los requerimientos que se incluyen en el proyecto y definen la funcionalidad que tendrá el producto a entregar al cliente. el alcance es lo primero que se determina. Esta variable es la que primero se negocia con el cliente.

### Tiempo - ¿Cuánto tiempo me llevaría?

Define la fecha de calendario en la cual se compromete a finalizar el proyecto y entregarle el producto resultante al cliente. En las metodologías ágiles, el tiempo es fijo.

### Costo - ¿Cuánto debería costar?

Define el costo de todos los recursos implicados en el desarrollo del proyecto. Esto incluye equipamiento, infraestructura, equipos, salarios, entre otros.

Cuando el alcance aumenta (mayor cantidad de funcionalidades), el tiempo y dinero (costos) necesarios también deben

## LÍDER DE PROYECTO

En un enfoque tradicional, el líder de proyecto realiza la toma de decisiones en su totalidad sin la inclusión de la opinión del equipo. El líder asigna las tareas que deben realizar los integrantes del equipo, y además, maneja todas las relaciones con todos los stakeholders del proyecto (clientes, gerencias, contratistas, stakeholders en general).

Es necesario una figura de líder de proyecto con las siguientes aptitudes:

Motivación: habilidad para alentar al personal, Organización, Innovación, Empatía, Comunicación, liderazgo.

También cuenta con conocimientos del producto, técnicas y herramientas.

Entre las responsabilidades del líder de proyecto se encuentran:

1. Definir el alcance del proyecto.
2. Identificar a los involucrados, recursos y presupuesto.
3. Detallar las tareas, estimar los tiempos y requerimientos.
4. Identificar y evaluar riesgos.
5. Preparar planes de contingencia.
6. Controlar y participar en las revisiones del proyecto.
7. Administrar el proceso de control de cambios.
8. Producir reportes de estado.

## EQUIPO DEL PROYECTO

Es un grupo de personas comprometidas con alcanzar un conjunto de objetivos

Entre sus características se encuentran:

1. Cuentan con conocimientos, habilidades técnicas.
2. Usualmente son un grupo pequeño, esto permiten lograr una comunicación efectiva entre los miembros del equipo.
3. Trabajan en forma conjunta. Para eso, se debe lograr un equipo maduro y estable.
4. Sentido de responsabilidad como una unidad y se focalizan en el cumplimiento de las metas grupales.

## STAKEHOLDERS

Son los interesados del proyecto, incluye el equipo de proyecto, el equipo de dirección, el líder de proyecto y el patrocinador.

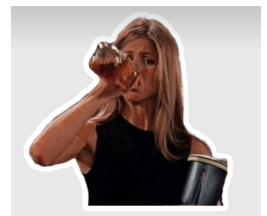
## PLAN DE PROYECTO

El plan de proyecto, (o también llamado plan de desarrollo de software en el ámbito del software), es un artefacto de gestión que cumple la función de “hoja de ruta” de un proyecto, en la cual se documentan todas las decisiones que se toman a lo largo del desarrollo.

En un plan de proyecto se establece qué se va a hacer (alcance del proyecto), cuándo se va a hacer (calendario), cómo se va a hacer (actividades o tareas para cubrir el alcance) y quién lo va a hacer (responsables de cubrir las actividades).

Responder a estas preguntas, implica las siguientes actividades:

1. Definición del alcance del proyecto.
2. Definición de proceso y ciclo de vida.
3. Estimación.
4. Gestión de riesgos.
5. Asignación de recursos.
6. Programación de proyectos.
7. Definición de métricas.
8. Definición de controles.



### 1. Definición del alcance del proyecto

El alcance del proyecto se mide contra el Plan de Proyecto (objetivos del proyecto).

El alcance del proyecto es la descripción clara y detallada de los objetivos, entregables, tareas y límites de un proyecto.

Tanto el alcance como el objetivo de un proyecto puede ser modificado a lo largo del proyecto.

Es importante aclarar que el alcance del proyecto es menor que el alcance del producto.

### 2. Definición del proceso y ciclo de vida

Al definir el proceso que se va a utilizar, se responde al cómo se va a desarrollar el proyecto.

También se debe tener en cuenta la complejidad del software a desarrollar (alcances y objetivos) para así poder adaptar el proceso definido que se elija al proyecto en el cual se está trabajando.

El ciclo de vida del proyecto es lo que define cuánto de cada tarea se debe hacer y en qué momento del desarrollo del proyecto hacerlo.

En metodologías tradicionales, se permite elegir cualquier tipo de ciclo de vida (cascada, iterativo e incremental, espiral, etc.), en cambio en metodologías ágiles esto no es posible (si o si tiene que ser iterativo e incremental).

### 3. Estimación

En un proyecto de desarrollo de software, se trata de estimar para predecir el valor de un elemento relacionado con el proyecto o con el producto, por ejemplo, el tiempo que va a llevar, qué costo va a tener el sistema, cuántas personas necesito para realizar el desarrollo, etc. En un enfoque tradicional, esta estimación de valores es realizada únicamente por el líder de proyecto. En la metodología tradicional, existe un orden definido y recomendado para realizar las estimaciones:

1. Tamaño del producto: para estimar el tamaño de un proyecto se utilizan los requerimientos tomados en la etapa de requisitos. Esta estimación es una de las más importantes a realizar, ya que el aumento o disminución del tamaño del producto, afecta de gran manera a otros aspectos

2. Esfuerzo: Esta estimación intenta responder cuántas horas personas lineales serán necesarias para construir el producto, previamente estimado su tamaño. el esfuerzo varía directamente con el tamaño de un proyecto

3. Calendario: la estimación del calendario del proyecto permite responder al cuándo se van a realizar las tareas definidas. Es necesario realizar esta estimación para poder darle una idea al cliente de cuánto tiempo se va a demorar la finalización del proyecto, y como en los demás aspectos, es necesario ya tener una buena estimación del tamaño y del esfuerzo que se requiere

4. Costos: de forma similar a la estimación de la duración del proyecto, la estimación de costos depende directamente del tamaño y del esfuerzo que se necesite para el desarrollo del software, por eso es lo último que se estima. De la misma manera, la estimación de costos es muy útil para darle una idea al cliente de cuál es el costo aproximado del proyecto.

### 4. Gestión de riesgos

Un riesgo es la probabilidad de ocurrencia de una pérdida o daño que afecte de forma directa al proyecto y pueda comprometer el incumplimiento de su objetivo.

Es de suma importancia identificar y definir los riesgos a los que está atado un proyecto, para así poder analizarlos, y para así definir a cuáles se debe atender y sobre cuáles se debe hacer un seguimiento.

Los podemos clasificar en:

1. Riesgos del proyecto: aquellos que alteran el calendario o los recursos del proyecto.

2. Riesgos del producto: aquellos que afectan la calidad o el rendimiento del software a desarrollar.

3. Riesgos organizacionales o de negocio: aquellos que afectan a la organización que desarrolla o que adquiere el software. Por ejemplo, un competidor que introduce un nuevo producto.

Para medir los riesgos se utiliza lo que se conoce como exposición al riesgo (probabilidad x impacto), la cual se obtiene como el producto entre la probabilidad de ocurrencia de la amenaza por el impacto que generaría si realmente ocurriera. Este valor permite organizar y gestionar los riesgos

Antes los riesgos identificados, es posible tomar 3 actitudes:

- Negación: hacer de cuenta que los riesgos no existen.
- Gestión Reactiva: esperar a que el riesgo ocurra y recién ahí ver que hacer.
- Gestión Proactiva: ir trabajando sobre el riesgo generado planes de mitigación y de contingencia.

## Gestión del riesgo

1. Identificación de riesgos: que suponen una mayor amenaza al proceso de ingeniería de software, al software a desarrollar o a la organización que lo desarrolla
2. Análisis de riesgos: para cada riesgo identificado se realiza un juicio acerca de la probabilidad y del impacto. El criterio dependerá de la experiencia de quien realice el análisis de riesgos.
3. Planeación del riesgo: para cada riesgo analizado, se definen estrategias para manejarlos. Estas estrategias consisten en considerar acciones a tomar para minimizar o evitar el impacto sobre el proyecto o desarrollar planes de contingencia.
4. Monitorización del riesgo y control: Este proceso se aplica en todas las etapas del proyecto.

### 5. Asignación de responsabilidades (o recursos)

La asignación de responsabilidades a personas permite asignar roles a los integrantes del equipo de trabajo.

Dentro de un mismo proyecto, una persona puede desenvolverse en más de un rol.

Para dejar explícitos los roles y quienes son las personas que los cumplen, generalmente se construye una tabla, la cual contiene la información de la persona, el rol y responsabilidades de esta en el contexto del proyecto.

### 6. Programación de proyectos (calendarización)

En esta etapa se trata de definir en detalle, cada tarea que debe ser cumplida en el desarrollo.

Cuando se habla de detallar las tareas, se hace referencia a descomponer el proceso de desarrollo en actividades refinadas a nivel de días, identificando quien la realiza, cuando debe realizarse y cuánto esfuerzo debe llevar en forma teórica.

### 7. Definición de métricas

Las métricas se definen como una medida numérica que aporta visibilidad sobre el avance o estado del proyecto, proceso o producto en un momento determinado del desarrollo. Al realizar la planificación del proyecto, es necesario definir de forma clara y no ambigua, cada una de las métricas asociadas a cada dominio en conjunto con la forma de calcularlas.

### 8. Definición de controles

En la planificación de los controles del desarrollo, se toma como base las métricas ya definidas, y verifica que todo se está haciendo en concordancia con lo establecido. En esta planificación se definen reuniones periódicas, reportes o informes necesarios, etc.

## PLANES DE SOPORTE

- Planes de Testing.
- Planes de Mantenimiento.
- Planes de subcontrataciones.
- Planes de calidad.
- Planes de capacitaciones.
- Planes de iteraciones (si el ciclo de vida es iterativo/incremental).

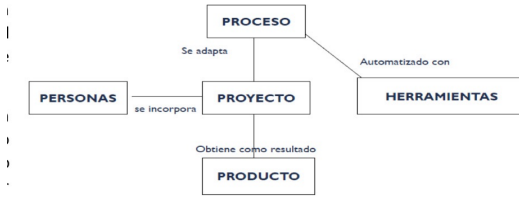
## CAUSAS DE FRACASOS DE PROYECTOS

- Fallas de toma de REQUERIMIENTOS (el 80%).
- Fallas al definir el problema.
- Planificar basado en datos insuficientes.
- La planificación la hizo el grupo de planificaciones.
- No hay seguimiento del plan de proyecto.
- Plan de proyecto pobre en detalles.
- Planificación de recursos inadecuada.
- Las estimaciones se basaron en "supuestos" sin consultar datos históricos.
- Nadie estaba a cargo.



## Vínculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.

Se dice que el **proceso**, automatizado con herramientas, se adapta al **proyecto**, al cual se incorporan **personas**, y de este se obtiene como resultado un **producto**. En el enfoque tradicional, previo a la definición del proyecto, es necesario determinar los objetivos y el ámbito del producto. Los objetivos identifican las metas globales del producto sin especificar como se van a lograr.



Para obtener como resultado un producto o servicio, es necesario que el proyecto adopte un proceso de desarrollo para poder definir qué actividades, métodos, prácticas y transformaciones se utilizarán, mediante las cuales se va a obtener el software. Dentro de cada proyecto de desarrollo de software, es necesario utilizar un proceso diferente dependiendo del tipo de producto que se desea desarrollar. Es por esto, que es necesario adaptar los procesos a cada uno de los proyectos en los cuales se trabaja. El proceso proporciona un marco conceptual en donde se establece un plan completo para el desarrollo del software.

Se debe definir el modelo de proceso a utilizar: secuencial, iterativo o recursivo. Luego las actividades del desarrollo del software se adapta al modelo elegido.

Un proyecto de desarrollo está integrado por un equipo de trabajo, dentro del cual deben estar los roles bien definidos,

Las personas son la parte más importante, debido a que el desarrollo de software es una actividad humano-intensiva.

En el desarrollo de un proyecto, se busca utilizar diversas herramientas que se adapten y permitan automatizar o facilitar las actividades que están definidas en el proceso

### No Silver Bullet No hay una solución exacta

- Contrastar progreso del hardware (algo tangible, construible, que se puede mejorar en su construcción), en comparación con el software (intangible, complejo inherentemente)
- Las balas de plata son aquellas soluciones que podrían hacer frente a la crisis del software: no existe un desarrollo de software que demuestre avances de progresión en la construcción del software en menos de una década.
- Dificultades que plantea:

**Esenciales:** todo aquello que trae aparejado el construir software. 4 aspectos la caracterizan:

#### Complejidad:

al crecer el tamaño del software, su complejidad crece de manera exponencial, no lineal

Esto conduce a muchos problemas: dificultad de comunicación entre miembros de equipos, deficiencias en el producto, errores en cumplimiento de fechas, excesos en costos, etc.

#### Conformidad:

el software debe realizar aquello que el cliente necesita, esto es: debe adecuarse a sus requerimientos.

#### Mutabilidad:

el software está sujeto a constantes cambios. A diferencia de objetos tangibles, el software muta y sufre cambios a lo largo del tiempo. En cambio, en muchos objetos esto no es así, ya que se reemplazan por nuevos productos.

#### Invisibilidad:

al ser algo no visualizable, se dificulta mucho trabajar con representaciones geométricas, Los grafos son una buena herramienta que permiten modelar muchas situaciones de un software.

**Accidentales:** tiene que ver con aquellos aspectos que dificultan la construcción del software, pero han sido solucionados a lo largo del tiempo. Esas soluciones son:

Lenguajes de programación de alto nivel

Tiempo compartido

Entornos de programación unificados

- **Resolución de dificultades accidentales**

- Programación orientada a objetos
- Inteligencia Artificial
- Sistemas expertos
- Programación automática
- Programación gráfica
- Verificación de programas:
- Environments and tools
- Estaciones de trabajo poderosas

- **Resolución de dificultades esenciales**

- Comprar, antes que construir: explorar en el mercado soluciones que se puedan conseguir para reemplazar lo que se quiera construir.
- Usar prototipos y refinar requerimientos
- Desarrollo incremental: Incrementar el software añadiendo funcionalidades mientras estas se prueban, usan y testean.
- Personas capacitadas: que sigan buenas prácticas. Esto plantea que tener en un equipo con buenos diseñadores, es tan importante como tener buenos líderes que dirijan ese equipo.

## Gestión de Productos de Software Lean-Agile



### Manifiesto Ágil/Filosofía Lean

Es un documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software.

#### Valores ágiles

El Manifiesto Ágil, expresa los siguientes valores. Esto es:

1. Individuos en interacciones, por sobre procesos y herramientas: es preferible un equipo de personas motivadas con un buen funcionamiento y conocimiento, donde la comunicación sea fluida, pero con herramientas pobres para su trabajo, que tener un equipo de individuos desmotivados o poco funcional con las mejores herramientas y procesos. debido a que el desarrollo de software es una actividad humano-intensiva, y los procesos ágiles intentan capitalizar las fortalezas de cada individuo
2. Software funcionando por sobre documentación extensiva: asegurando que se tengan versiones estables, incrementales y mejoradas del software al finalizar cada iteración. tras cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación de ellos. se debe generar la documentación cuándo sea necesario. No plantea “no generar documentación”
3. Colaboración con el cliente por sobre negociación contractual: esto permitirá no sólo tener una mejor noción de aquellos requerimientos de negocio que necesita el cliente, sino también que el equipo ágil se integre a todas las partes interesadas en el software, para que en conjunto puedan trabajar y lograr los mismos objetivos. Esto también reduce la dependencia de documentación extensiva ya que cada miembro del equipo participa en la toma de decisiones. se trata de tener a un cliente lejano que no sabe lo que sucede durante el desarrollo, si no tener a uno que esté comprometido con cada entrega, sobre todo al momento de probarlo y dar el feedback. A veces las negociaciones contractuales imponen ciertas restricciones que impactan de manera negativa
4. Responder a cambios por sobre seguir un plan: esto se plantea debido a que es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan, y a su vez, con el paso del tiempo surgirán nuevas necesidades. Esto está ligado al entorno cambiante , por lo que un equipo ágil está abierto a los cambios

## Principios del Manifiesto Ágil

1. Nuestra mayor prioridad es satisfacer al cliente: esto está ligado al valor 3 y 4, debido a que un equipo ágil desarrolla y entrega funcionalidades en el orden especificado por el Product Owner, quien determina la prioridad de cada funcionalidad, de manera que en cada incremento del producto, dé al cliente el mayor valor posible. Esto está soportado por el hecho de trabajar con User Stories, las cuales plantean necesidades que tienen un valor significativo para los usuarios del software. El Product owner es el encargado de escribir las User Stories.
2. Aceptar que los requisitos cambien, incluso en etapas tardías de desarrollo: ligado a los valores 3 y 4, donde se plantea que el equipo ágil está abierto al cambio, al no seguir un plan estructurado que se confecciona en etapas temprana. Esto permite que el equipo responda siempre ante las necesidades cambiantes de los clientes y/o usuarios.
3. Entregar software funcional frecuentemente: esto se relaciona al valor 2, donde es imprescindible que un equipo ágil al finalizar cada iteración, haya sido capaz de transformar uno o más requerimientos en código desarrollado completamente, testeado y potencialmente desplegable. Se debe asegurar que cumpla con lo que el usuario/cliente necesita, de forma que se le entregue el mayor valor posible tras cada iteración.
4. Técnicos y no técnicos trabajando juntos durante todo el proyecto: valores 1 y 3, y lo que plantea es la gran importancia de la participación del rol conocido como "Product Owner", en conjunto con el equipo de desarrollo de software. Este rol, que generalmente ocupado por una persona del "lado del cliente", debe ser alguien capaz de tomar decisiones sobre el producto, y que además sea capaz de representar el interés y necesidades los usuarios a los que estará destinado el desarrollo del software.
5. Desarrollamos proyectos en torno a individuos motivados: este principio está ligado al valor 1, donde la filosofía asume que aquellos individuos que participan en un proyecto de desarrollo del software deben estar lo suficiente motivados y comprometidos con los objetivos de este. Los participantes de un proyecto se deben ver entre sí como a un único equipo con objetivos comunes.
6. El método más eficiente de comunicar información es con conversaciones cara a cara: ligado con el valor 1, este principio tiene un sustento psicológico, el cual expresa que las palabras expresadas como sonido o texto, sólo representan un bajo porcentaje de la comunicación, siendo que la mayor parte está ligada a gestos, expresiones faciales, movimientos corporales, etc.
7. La mejor métrica de progreso es el software funcionando: ligado con el valor 2, donde el manifiesto ágil plantea que la mejor forma de demostrar el avance en un proyecto de software es a través de la entrega de software funcional y de valor para el cliente.
8. Los procesos ágiles promueven el desarrollo sostenible: ligado con los valores 2 y 4, esto apunta a la estabilidad en equipos de trabajo a lo largo del tiempo, lo cual se traduce en una mejora continua de relaciones y formas de trabajo del equipo. Mantener un equipo estable a lo largo del tiempo permite que el equipo alcance una madurez tal, en la cual es mucho más fácil realizar estimaciones. Por ejemplo, el tiempo en el que el equipo desarrolla una cantidad de US.
9. La atención continua a la excelencia técnica y al buen diseño mejora la agilidad: esto se relaciona al valor 2, y manifiesta que la calidad del producto no es un aspecto negociable, entregando al cliente código de calidad.
10. La simplicidad es esencial: ligado a los valores 2 y 3, este principio apunta a que, como equipo de desarrollo, nos centremos en aquellas necesidades que realmente aporten un valor al cliente, y no perder el tiempo en cosas que no satisfacen requerimientos de los usuarios. No hacer cosas que el cliente no pidió.
11. Las mejores arquitecturas, diseños y requerimientos surgen de equipos autoorganizados: ligado al valor 1, este principio expresa que en los equipos de desarrollo ágiles los integrantes del equipo deben ser proactivos, aportando soluciones, herramientas, etc. que sean útiles al equipo.
12. A intervalos regulares, el equipo evalúa su desempeño y ajusta la manera de trabajar: ligado a los valores 1 y 4, este principio se basa en los dos pilares de los procesos empíricos (inspección y adaptación), los cuales permiten a los equipos que cada determinados períodos de tiempo evalúen distintos aspectos de su desempeño, y adaptar aquellos aspectos que sean necesarios corregir, mejorar o incorporar al equipo.



## Filosofía Ágil

El objetivo primordial es entregar software de forma frecuente en un entorno cambiante. Estas metodologías se utilizan en entornos con gran variabilidad de requerimientos, involucrando al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación.

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde se obtiene una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema.

Usualmente los incrementos se crean cada 2 o 3 semanas y se ponen a disposición del cliente, ofreciendo una rápida retroalimentación que permiten una inspección y adaptación sobre los requerimientos cambiantes. Por otro lado, se reduce la cantidad de documentación con el uso de comunicaciones frecuentes e informales con el cliente

### Pilares del empirismo

**TRANSPARENCIA:** la transparencia nos permite crecer como equipos y transformar el conocimiento que cada miembro del equipo tiene implícito en conocimiento explícito, es decir, que pase a formar parte del conocimiento de todo el equipo. Todo lo que desarrollemos y nos pase dentro del equipo, pasa a ser parte de todos. La inspección sin transparencia genera engaños y desperdicios.

**INSPECCIÓN:** Se realiza por todo el equipo y permite detectar varianzas o problemas potencialmente indeseables, se debe inspeccionar el progreso hacia los objetivos acordados. Es el momento de aprender cosas nuevas que van surgiendo del trabajo que el equipo va realizando. La inspección sin la adaptación se considera inútil.

**ADAPTACIÓN:** Se trata de la mejora continua, es la capacidad de adaptarse en función de los resultados de la inspección. El equipo debe adaptarse en el momento que aprenda algo nuevo por medio de la inspección.

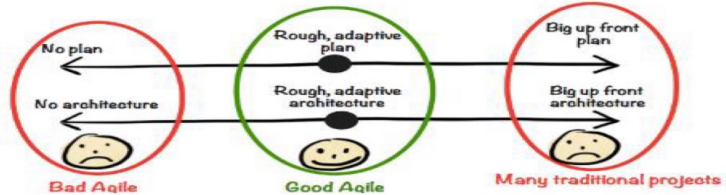
### ¿Cuándo es aplicable ágil?

Primero es importante mencionar que ágil no es una metodología o un proceso, ni tampoco un framework completamente definido, sino que es una ideología con un objetivo definido de valores y principios que guían el desarrollo del producto. definen prácticas de cómo hay que organizarse, es decir, cómo debe organizarse el equipo.

las metodologías ágiles son aplicables en el contexto de lo complejo , en el cual se ubican los nuevos productos sobre los cuales el entendimiento y la certeza que se tiene es parcial.

### Don't go overboard with Agile!

Ser ágil no es ser indisciplinado, no hay que recaer en lo que la imagen representa.



### ¿Por qué ir a ágil?

- Acelera entregas de productos de valor para el cliente.
- Permite adaptarse fácilmente a cambios de prioridades y requerimientos.
- Permite alinear los objetivos organizacionales y de TI.
- Reduce riesgos de proyecto.
- Permite una mejor visibilidad del proyecto.
- Mejora las relaciones en el equipo.
- Reduce costos de proyecto.

## El triángulo ágil

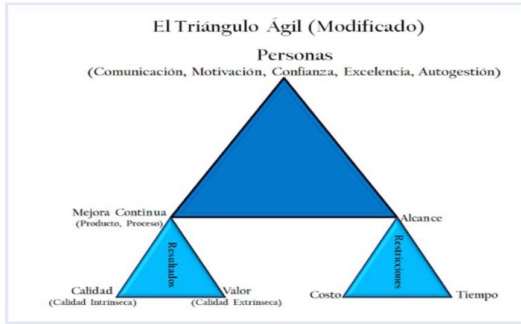
El triángulo ágil apunta a que lo que realmente tiene **valor** que es el software funcionando para el cliente, por lo que se centra más en la **calidad** de producto. El triángulo ágil (modificado) apunta a un **producto** de software que pone a las **personas** en el lugar más importante.

- Las **personas** son lo más importante, donde se prioriza que haya comunicación, motivación, confianza, excelencia y autogestión.

- Debajo encontramos el pie, en el cual tenemos la **mejora continua**. Es el triángulo de resultados, teniendo como pie a la **calidad** (la cual es calidad intrínseca) y al **valor** (calidad extrínseca).

- Del otro lado de la base podemos encontrar el **alcance**, el cual nos da cuáles serían las **restricciones** de nuestro producto, y encontramos el **costo** y el **tiempo**, los cuales suelen ser grandes limitantes a la hora de desarrollar software.

Debemos hacer foco en entregarle al cliente un producto de calidad que se vaya mejorando y evolucionando en forma continua.



## Filosofía Lean

Es una filosofía o enfoque más viejo que agile, busca maximizar el valor generado al cliente con el mínimo consumo de recursos. Esto permite reducir los esfuerzos en crear artefactos que no agreguen valor a un producto, concentrando los esfuerzos en los aspectos esenciales.

La filosofía "Lean", permite atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente final quiere, cuando lo quiere, dónde lo quiere, a un costo mínimo y precio justo. Siendo el cliente quien determina si el servicio o producto que la empresa entrega tiene valor o no.

Para esto utiliza el método Just in Time, el cual permite el intercambio del producto entre fases del proceso de desarrollo o a la entrega del producto final

## 7 Principios Lean

### 1. Eliminar desperdicios

Es quizás el más importante. Debemos tener un proceso que sea eficiente y que sume valor en todos los pasos del proceso, todo paso o parte que no contribuye a la generación de valor produce un desperdicio.

Los 7 desperdicios de Lean

*Características extras* [producir productos extras]

*Trabajo a medias* [acumulación de stock]

*Proceso extra*: los pasos extras que no aportan valor al producto. El objetivo es identificarlos y eliminarlos.

*Búsqueda de información*: Esto termina siendo un problema importante y difícil de manejar si no se implementa la disciplina SCM.

*Defectos*: Son errores que se trasladan de una etapa a otra etapa posterior en la cual se introdujo

*Esperas* [costos de logística]: tiempo de espera que una persona tiene que pasar para poder realizar su trabajo.

*Cambios de tareas (Multitasking)*: Esto genera una pérdida de tiempo en prepararnos para realizar cada tarea

### 2. Amplificar el aprendizaje

Tiene que ver con la transparencia y transformación de conocimiento implícito en explícito, lo cual fortalece al equipo de trabajo. Los individuos intercambian sus experiencias y conocimientos para contribuir con el aprendizaje colectivo.

### 3. Embeber la integridad conceptual

La calidad del producto no se negocia y la simplicidad es un factor clave: el producto tiene que satisfacer las necesidades del cliente, esa es la principal prioridad y cada decisión se alinea con esto. Para esto se deben encastrar todas las partes del producto que tengan coherencia y consistencia para hacer al producto más íntegro

### 4. Diferir compromisos hasta el último momento responsable

Esto apunta a postergar la toma de decisiones lo suficientemente como para poder tener la mayor información necesaria para tomar esa decisión, pero a la vez, antes de que ese momento sea muy tarde

## 5. Dar poder al equipo

Otorgar libertad de acción y poder de decisión al equipo. Para ello, el equipo debe ser multifuncional y sus miembros deben estar capacitados y motivados. implica un compromiso por parte de los miembros del equipo, para desempeñarse y cumplir con lo pactado de forma completa y correcta.

## 6. Ver el todo

En Lean se busca tener una visión holística, que permita asociar y comprender el todo, porque la idea es que los objetivos particulares de las partes estén alineados con los objetivos globales.

## 7. Entregar lo antes posible

La idea es entregarle al cliente software funcionando lo más rápido posible y que este producto sea útil para él. Se habla de entregarle software antes de que las necesidades de negocio cambien.

Para esto Lean propone acotar ciclos de desarrollo y entregar rápidamente incrementos pequeños de valor, lo que permite salir pronto al mercado con un producto mínimo que sea.

## Relación Lean-Ágil

Lean es previo al Manifiesto Ágil. Algunos principios de Agile heredan de los fundamentos de Lean. Ambos están orientados al cliente, a proveer el máximo valor posible. Para esto, entienden que los individuos de los equipos deben estar motivados. La flexibilidad para adaptarse a los cambios y ofrecer valor al cliente, es un denominador común de ambos enfoques. Otro aspecto que tienen en común es generar productos de calidad y de mejora continua.

## Requerimientos en ambientes ágiles

Lo más difícil del software es decidir que software queremos construir, ya que los proyectos de software fracasan debido a que no se cumplen con las expectativas del cliente. Este problema se advierte en la crisis del software.

Probablemente una de estas razones es no asimilar la complejidad que supone hacer software y que la etapa de relevar requerimientos suele ser subestimada.

También, es muy común que haya problemas de comunicación entre los clientes y el equipo. O que soliciten funcionalidades y fechas de entrega que los desarrolladores no puedan cumplir.

Lo óptimo es que haya una comunicación de ambas partes, ya que los proyectos suelen fracasar si no.

## Fundamentación de los requerimientos ágiles

El fondo de la cuestión está en los principios y valores explicitados en el manifiesto ágil, fundamentalmente la entrega frecuente y temprana de software valioso y la posibilidad de recibir cambios de requerimientos aun en etapas finales. La base de los requerimientos en ambientes ágiles tiene que ver con la posibilidad de comenzar a construir el producto cuando el producto no está completamente definido.

Los requerimientos ágiles se basan en 3 pilares.

- *Usar el valor para construir el producto correcto*: Lo único importante es dejar contento al cliente con software funcionando que le sirva, por lo que, el foco de este enfoque de gestión ágil apunta a construir valor de negocio.

- *Determinar que es "sólo lo suficiente"*: hace referencia a que los requerimientos deben ser encontrados de a poco, no buscar todos los requerimientos desde un inicio y quedarnos solo con eso. Se tiene que empezar con lo mínimo necesario y generar realimentación

- *Usar historias y modelos para mostrar que construir*: Se busca trabajar con un referente que esté del lado del negocio, que tenga el conocimiento del producto y que trabaje junto con el equipo.

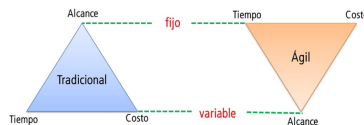
El enfoque de requerimientos ágiles debe estar adherido al manifiesto ágil, es decir, a cumplir con los valores y con los principios que el manifiesto plantea.

## La triple restricción – enfoque ágil

En metodologías ágiles, la visión de la triple restricción :

- Tiempo y Costo suelen fijarse
- Alcance se vuelve variable y se ajusta de manera flexible, priorizando lo que da más valor al negocio.

Esto significa que en lugar de prometer "entregar absolutamente todo en determinada fecha con un presupuesto fijo", se prioriza entregar el máximo valor posible dentro de ese tiempo y costo.



## BRUF – Big Requirements Up Front

A lo que se apunta con este concepto de BRUF es a notar que, en una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades.

Además, es prácticamente imposible conocer al 100% todas las necesidades de los usuarios, ya que ni ellos saben concretamente qué necesitan. Por otro lado, esas necesidades que tienen los usuarios con el tiempo probablemente irán cambiando, con lo cual, la priorización es un factor fundamental en esa etapa, para que se puede entregar una nueva versión del producto que sea de valor para el cliente.

## Product Backlog

El Product Backlog es una lista priorizada de características y requerimientos del software. Es el Product Owner quien se encarga de definir esta lista ya que es él quien tiene en claro cuáles son sus necesidades y requerimientos principales. El

Product Owner se va a encargar de decidir qué requerimientos necesita primero según el valor para el negocio.

## Requerimientos Just in Time

Es la práctica de definir los requerimientos de manera progresiva, En lugar de documentar todo el sistema al inicio (como en enfoques tradicionales), se van detallando los requerimientos cuando el equipo está cerca de implementarlos.

Se empieza con una visión del producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación.

## Fuente de requerimientos

Los requerimientos ágiles, basados en el principio de que el mejor medio de comunicación es el cara a cara, promueven que los requerimientos del software se obtienen conversando cara a cara con el cliente.

Los agilistas no escriben una ERS como en los métodos tradicionales, pero lo compensan con la disponibilidad del cliente para resolver las dudas.

## Momento de la captura de requerimientos:

En el enfoque tradicional, los requerimientos son definidos al principio del proyecto. En el enfoque ágil, los requerimientos son relevados continuamente durante todo el proyecto.

## Contenedor y persistencia de los requerimientos

El enfoque tradicional, la ERS contiene los requerimientos a lo largo del ciclo de vida del producto todo el tiempo. En caso de que surjan cambios, es necesario actualizar la ERS.

En el enfoque ágil, se utiliza al Product Backlog como un contenedor transitorio de los requerimientos.

## Tipos de requerimientos

La parte baja de la pirámide, corresponde a los requerimientos del Dominio de la solución (esto no le interesa al cliente). Lo que está por encima, es decir, Requerimientos de Usuario y Requerimientos de Negocio, corresponden al Dominio del Problema, y es lo que al cliente sí le interesa.

A nivel de la gestión ágil de requerimientos se trabaja con:

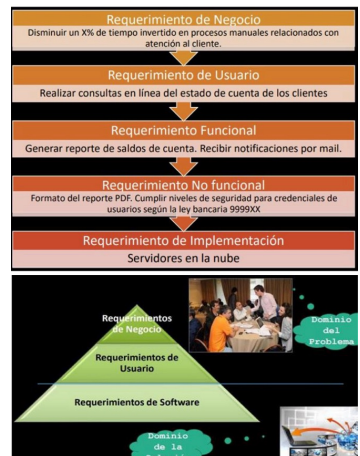
- Requerimiento de negocio
- Requerimiento de usuario

Las User Stories son requerimientos de usuario alineados a un requerimiento de negocio. Al usuario le interesa el requerimiento de negocio.

El valor de negocio es el requerimiento de negocio, pero el requerimiento de usuario es el medio por el cual vamos a lograr ese valor de negocio.

¿Qué debemos asumir?

- Todo el tiempo va a haber cambios en los requerimientos.
- Stakeholders: Debemos saber quiénes son los involucrados y los involucrados son todos los que tiene algo que decir respecto del producto.
- Siempre se cumple eso de que: “El usuario dice lo que quiere cuando recibe lo que pidió”.
- No hay técnicas ni herramientas que sirvan para todos los casos. Debemos detectar para cada situación cuál es la herramienta más adecuada.
- lo importante es entregar, un resultado, una solución de “valor”.







1- LA PRIORIDAD ES SATISFACER AL CLIENTE A TRAVÉS DE RELEASES TEMPRANOS Y FRECUENTES (2 SEMANAS A UN MES)



2- RECIBIR CAMBIOS DE REQUERIMIENTOS, AUN EN ETAPAS FINALES



4- TÉCNICOS Y NO TÉCNICOS TRABAJANDO JUNTOS TODO EL PROYECTO



6- EL MEDIO DE COMUNICACIÓN POR EXCELENCIA ES CARA A CARA



11- LAS MEJORES ARQUITECTURAS, DISEÑOS Y REQUERIMIENTOS EMERGEN DE EQUIPOS AUTOORGANIZADOS

## User Stories

Son una técnica para trabajar requerimientos de usuario en ambientes ágiles. La US contiene una descripción corta de una funcionalidad que se espera del producto.

Funciona como un recordatorio para el equipo de desarrollo y el Product Owner. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Para eso está el manual de usuario. Las US son escritas por el Product Owner.

Internamente, las User Stories se consideran verticales dentro del producto.

Se dice que son multipropósito, debido a que cumplen diferentes funciones en el desarrollo de un software:

- Representan una necesidad de usuario, de forma no detallada.
- Conforman una descripción del producto.
- Son utilizadas como ítems de planificación.
- Se utilizan como recordatorios de futuras conversaciones acerca del producto que se desea construir.
- Junto con el código ejecutable, permiten documentar el producto.

### Partes de la User Story

Está compuesta por tres partes:

1. Tarjeta: es una descripción de la historia, utilizada para planificar y como recordatorio. Tiene una sintaxis particular que es la siguiente:

Como <<nombre del rol>> yo puedo <<actividad>> de forma tal que <<valor de negocio>>

- Nombre del rol: representa quien está realizando la acción o quien recibe el valor de la actividad.
- Actividad: representa la acción que realizará el sistema.
- Valor de negocio: es la más importante y representa el por qué es necesaria la actividad.
- Frase verbal: no es obligatoria, es una forma corta de referenciar la US.
- Criterios de aceptación: son diferentes criterios que permiten definir los límites de la U.S. para que el equipo tenga una mejor visión de esta.
- Pruebas de aceptación: acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles de la User.

2. Conversación: Es la parte más importante de la US, ya que explicita la comunicación entre el Product Owner y el Equipo de desarrollo que permite compensar la falta de especificación y detalle.

También es posible dejar la conversación persistente en grabaciones pero De todas maneras, la conversación se materializa en la tarjeta y en las pruebas de aceptación.

En simples palabras, son discusiones acerca de la historia que sirven para desarrollar los detalles de la historia.

3. Confirmación: Definición de un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó. El PO decide si las pruebas de aceptación son válidas.

### Ventajas

- Prioriza la comunicación verbal por sobre la escrita
- Son comprensibles tanto para el cliente/usuario (Product Owner) como para el equipo de desarrollo
- Tienen el tamaño justo para permitir planificar.



- Permiten el desarrollo iterativo al dividir todas las funcionalidades del producto en US.
- Permiten desestimar los detalles, hasta que se logre una mejor comprensión la necesidad de los usuarios.

#### Desventajas

- En proyectos grandes, es difícil entender la relación entre U.S
- No son apropiadas para equipos demasiado grandes

### Product Backlog y las User Stories

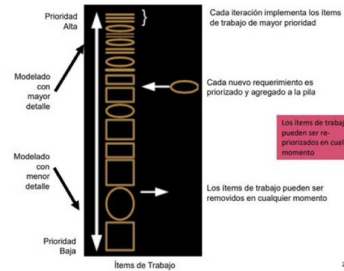
Es una pila de PBIs (Product Backlog Items), En la parte más alta de la pila, se ubican las U.S. de mayor prioridad, y las de menor prioridad al fondo. En cualquier momento se puede cambiar esa priorización, según cómo lo indique el Product Owner, quien es el encargado de realizar el proceso de priorización.

En cualquier momento del proyecto se van agregando nuevas U.S. y actividades a realizar, según se vayan detectando nuevos requerimientos.

Debido (agregar y quitar ítems), nunca encontraremos el 100% de los requerimientos de usuario en el Product Backlog.

Porciones verticales

Las US son porciones verticales.



#### Tamaño

Es recomendable tener una gran cantidad de historias con tamaños relativamente pequeños, que tener historias muy grandes.

El tamaño de las U.S. se mide en Story Points.

Hay 3 aspectos que determinan el tamaño de una U.S.:

- Complejidad
- Esfuerzo: es el tiempo en horas de trabajo que requerirá la implementación.
- Incertidumbre
- Técnica: cuando se tiene dudas en el dominio de la solución
- De negocio: cuando hay incertidumbre respecto a cómo interactuará el usuario con el sistema



### Story Points

Son unidades de medida relativas, las cuales miden el tamaño de un producto. Esto se debe a que estimar basándonos en el tiempo, conduce a muchos errores.

#### Niveles de abstracción

1. User Story
2. Épica: es una User muy grande. Si la User no entra en la iteración es considerado una épica. Llegado el momento de su implementación, se deberán dividir en historias de usuario más pequeñas
3. Temas: son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.

### Modelado de Roles

Se utilizan las tarjetas de rol de usuario, aquí se anotan las características en general que va a tener el usuario.

#### Técnicas adicionales

*esto no se q puta es*

1. Personas:
2. Personajes extremos
3. Proxies (Usuarios Representantes)

#### Criterios de aceptación

Es una nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, tiene que servirnos a nosotros para saber si lo que implementamos es correcto o no. El PO la va a aceptar si respeta los criterios de aceptación.

¿Cuáles son los criterios de aceptación buenos?

- Definen una intención, no una solución. Ejemplo: El usuario debe elegir al menos una cuenta para operar.
- no es necesario que se escriba cada detalle.

¿Y los detalles, a dónde van?

Detalles van en :

- Documentación interna de los equipos
- Pruebas de aceptación automatizadas.

### Pruebas de aceptación

Se prueban escenarios exitosos y escenarios que fallen. En las pruebas de aceptación se encuentran las más importantes, ya que no es posible escribir todas las posibles pruebas de aceptación.

El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen.

Expresan detalles resultantes de las conversaciones entre clientes y desarrolladores; suelen usarse para completar detalles de la US. Es importante mencionar que las mismas deben escribirse antes que la programación empiece

### DoR (Definition of Ready) – Definición de Listo

Es una medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo.

Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model.

#### INVEST Model

Es un modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la **definición de Ready** y poder ingresar en una iteración de desarrollo.

- Independiente: no debe existir una dependencia con otra User para desarrollarla
- Negociable: deben servir a modo de recordatorio para negociar aquellos aspectos en un futuro
- Valuable: Una US debe ser valorable para los clientes o usuarios, deben aportar valor de negocio
- Estimable: la US debe contener la cantidad de información suficiente para estimar el tamaño, la complejidad y el esfuerzo necesario para realizarlo.

### DoD (Definition of Done) – Definición de Hecho

Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no. momento en que una U.S. cumple con todo lo pactado en esta Definición de Hecho

### Spikes

Son un tipo especial de US que se usa cuando existe una incertidumbre, la cual imposibilita que pueda ser estimada y por lo tanto no cumple con la definición de listo, Una vez resuelta la incertidumbre, la Spike se convierte en una o más US.

#### Spikes técnicas

Utilizadas para evaluar diferentes enfoques técnicos y tecnológicos en el dominio de la solución.

#### Spikes funcionales

Utilizada cuando hay cierta incertidumbre respecto de cómo el usuario interactúa con el sistema

### Investment Themes (Temas de Inversión)

Representan un conjunto de iniciativas o propuestas de valor que conducen la inversión de la empresa en sistemas, productos, aplicaciones y servicios con el objetivo de lograr una diferenciación en el mercado y/o ventajas competitivas.



## Estimación en ambientes ágiles

Estimar es el proceso de obtener una aproximación con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto.

Según McConnell una estimación en el contexto del software es una predicción de cuánto tiempo durará o cuanto costará un proyecto. Es considerada una de las actividades más complejas en el desarrollo de software.

### Errores en las estimaciones

1. Información imprecisa acerca del software a estimar.
2. Demasiado caos en el proyecto
3. Imprecisión generada por el proceso de estimación.
4. omitir actividades necesarias para la estimación
5. Se olvidan que planear no es estimar y estimar no es planear.
6. A mayor diferencia entre lo planeado y lo estimado, mayor el riesgo.
7. Las estimaciones no son compromisos.

### ¿Por qué estimamos?

- Para predecir completitud.
- Para administrar riesgos.

Las estimaciones requieren:

- Experiencia.
- Acceso a buena información

A medida que avanza el proyecto, las estimaciones son más certeras, dado que se cuenta con mayor información sobre la cual realizar las estimaciones.

Generalmente Al inicio del desarrollo hay una estimación errónea de 2 a 4 veces más

### Proceso de estimación

En "Plan de proyecto " arriba

### Métodos utilizados para estimar

Se recomienda utilizar diferentes métodos de estimación y luego contrastar (comparar).

#### 1. Basados en la experiencia

a. Datos históricos: consiste en recolectar datos básicos de otros proyectos para ir generando una base de conocimientos que sean de utilidad para futuras estimaciones,

b. Juicio experto: es uno de los más utilizados, el problema se presenta cuando se utiliza como única técnica de estimación el juicio experto puro. Existen dos técnicas para aplicarlo:

i. Juicio experto puro: un experto estudia las especificaciones y realiza una estimación. Esta estimación se basa fundamentalmente en la experiencia del experto.

ii. Juicio Wideband Delphi: es similar al anterior pero grupal, un grupo de personas se reúne con el objetivo de converger a una estimación común. Este método es tomado como base para el Poker Planning.

#### 2. Analogía

Consiste en tomar a un proyecto e ir comparando factores, por ejemplo:

- Tamaño: es menor o mayor al proyecto anterior.
- Complejidad: más complejo de lo usual.
- Usuarios: si hay más usuarios habrá más complicaciones

#### 3. Otros

a. Basados exclusivamente en los recursos: consiste en analizar el personal que se tiene y de cuánto tiempo se dispone del mismo, haciendo las estimaciones exclusivamente en función de este factor.

b. Basados en la capacidad del cliente o exclusivamente en el mercado: ponen el foco en el cliente y realizan las estimaciones en función de su capacidad de pago.

c. Basados en los componentes del producto o en el proceso de desarrollo

d. Métodos algorítmicos: se ejecutan algoritmos, a pesar de ser un cálculo matemático, puede no ser certero

## Problemas de estimaciones tradicionales

- Estimar demasiado pronto.
- Resistencia a las reestimaciones.
- Estimaciones que buscan precisión.
- Estimaciones hechas por gente distinta a la que hace el trabajo.
- Estimaciones que pretenden ser absolutas (inflexibles).

## Estimaciones en ambientes ágiles

Son presunciones numéricas asociadas a una probabilidad de que cierto evento ocurra. La filosofía ágil plantea que quien estima, es quien debe hacer el trabajo, con lo cual es el equipo de desarrollo quien realiza las estimaciones, de forma colectiva (menos el Product Owner, no participa en estimaciones), haciendo que ese proceso de estimar se vuelva lo más preciso posible

No es una instancia única, ya que a medida que se avance el proyecto, más información vamos a tener y más eficiente van a ser nuestras estimaciones.

En el enfoque ágil, el esfuerzo destinado para estimar es bajo. el tiempo de estimación se define y asigna con anterioridad para no utilizar más esfuerzo del necesario.

Existe una serie de consideraciones importantes:

- Son medidas relativas no absolutas.
- No es una medida basada en el tiempo.
- Las personas no saben estimar en términos absolutos, somos buenos comparando.
- Comparar es generalmente más rápido.
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual.
- Se emplea mejor el tiempo de análisis de las Stories.



Para las estimaciones se deben tener en cuenta: complejidad, esfuerzo e incertidumbre.

## ¿Qué se estima?

Lo que se estima en ambientes ágiles, es solo el tamaño de las US (o Features) usando como unidad de medida los Story Points, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo. Se utilizan comúnmente series numéricas como la serie de Fibonacci.

Es importante tener en claro que no se estima tiempo, ya que la duración de los Sprints es fija

*Y sigue diviendo lo mismo una y otra vez*

## Tamaño vs. Esfuerzo

Como el esfuerzo (horas de trabajo) es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario. La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo. Por otro lado, no se debe confundir esfuerzo con calendario, ya que yo puedo decir que puedo tener un trabajo para el viernes, pero en el medio pueden suceder millones de cosas que evitan que cumpla mi objetivo.

## Velocidad

Es una métrica del progreso de trabajo de un equipo. Se calcula (no se estima) sumando el número de Story Points asignados al estimar una US.

Si la US se completó parcialmente, no se cuentan sus Story Points en esa iteración. Esto genera que la velocidad del equipo baje en esa iteración, pero en la siguiente, aumentará.

## Reestimación

Estimar nuevamente una US es aconsejable sólo cuando la opinión acerca del tamaño relativo de una US ha cambiado. No se debería reestimar porque no se detecta un progreso como el esperado (velocidad del equipo), sino que se debe dejar que la velocidad a lo largo de sucesivas iteraciones equilibre esas inexactitudes.

## Planificación

En ambientes ágiles, es necesario que quien planifica el proyecto de desarrollo de software, sea capaz de analizar el contexto del proyecto cada cierto período y hacer aquellos ajustes que sean necesarios. Lo importante de planificar es el proceso en sí.

## Planificación de Release

Este proceso se da al inicio del proyecto, donde se analiza y determina qué US, épicas o Temas serán desarrollados en una nueva entrega (Release) del producto. Este plan se va actualizando mientras el proyecto transcurre. Una vez que se priorizan las US a implementar, se deriva la duración de la Release

### Planificación de Iteración

Este proceso se da al comienzo de cada iteración, donde el PO identifica aquellas US de mayor prioridad que deberán ser implementadas en la iteración. Esto se hace teniendo en cuenta el tiempo y la velocidad de trabajo del equipo . Durante esta planificación, se discute sobre las tareas que serán necesarias para transformar una US en código funcional y testeado.

### Planificación del día

Este proceso permite coordinar y sincronizar el trabajo del equipo, normalmente en reuniones diarias. Allí sólo se discuten las tareas a realizar durante el día, haciendo foco en las actividades que deberán ser desarrolladas para lograr cumplir con una tarea específica.

### **Tamaño**

Dado que el tamaño es lo que se compara, lo definimos como la medida de cuán compleja y grande es una US.

#### Formas de estimar el tamaño

Existen diferentes escalas para estimar el tamaño: una de ellas es la Serie de Fibonacci

Una vez definida la escala, no se cambia. En caso de cambiarla, también se debe cambiar el patrón de medición.

### **Story Point**

Es una unidad de estimación que especifica la complejidad, incertidumbre y esfuerzo propio del equipo respecto a una US en términos relativos respecto a otra US. Es lo que le da la idea del "peso" de cada US,

#### **¿Para qué se estima?**

1. En un primer momento para determinar cuántas Users se puede comprometer el equipo de trabajo a terminar en una iteración.
2. Al finalizar la iteración, contamos los puntos de historia correspondiente a las Users aceptadas por el Product Owner y obtenemos la velocidad del sprint, la cual es una de las métricas fundamentales en ambientes ágiles.

#### **¿Cómo se estima la duración del proyecto?**

Si la estimación se realiza utilizando Story Points para medir la complejidad relativa de las User Stories, para determinar la duración de un proyecto

### **Poker Planning o Poker Estimation**

Es una técnica de estimación en ambientes ágiles publicada por Mike Cohn. Resulta de la conjunción de diferentes métodos, y asume que las personas que desarrollan el producto deben ser aquellas que estimen. el Product Owner participa en la planificación, pero no realiza estimaciones.

### Escala

Dado que la complejidad en el software incrementa de forma exponencial, se decide adoptar la serie de Fibonacci como escala para la determinación de los puntos de historia.

1. Valor 1 o 1/2: son los valores más bajos y se aplican a una funcionalidad pequeña.
2. Valor 2- 3: funcionalidad pequeña a mediana.
3. Valor 5: funcionalidad media.
4. Valor 8: funcionalidad grande, donde nos preguntamos si es posible dividir.
5. Valor 13 o más: necesariamente hay que dividir ya que no ingresa en el sprint.

### **Procedimiento**

Se define una lista de actividades, módulos, casos de uso, Users stories, etc.

Se acuerda la más simple y Esta User es utilizada como referencia para comparar.

Cada miembro elige una carta con el valor de la estimación del tamaño de la U.S. (comparado con la U.S. canónica (la más simple) y teniendo en cuenta los 3 aspectos nombrados) y se espera a que todos decidan.

Utilizando juicio experto, cada integrante del equipo de desarrollo expone su valor de estimación de forma simultánea, por medio de las cartas, dándolas vuelta a las que estaban boca abajo. En caso de que la estimación difiera, el mayor y el menor estimador justifican sus estimaciones para conocer sus opiniones.

Luego el resto del medio aportan sus opiniones. Siempre la escucha es activa, para poder aprender.

Si las estimaciones no convergen, entonces se llega a una estimación intermedia o promedio.

#### **Ventajas**

- Permite opiniones de todo el equipo, lo cual enriquece la estimación.
- Al generar debates y discusiones, permite lograr una mejor comprensión del dominio
- Estudios demuestran que reunir estimaciones individuales conducen a mejores resultados, que grupal.
- Es una actividad que resulta divertida para el equipo.



#### **Estimaciones en Lean**

En Lean, el cual es más extremo que las metodologías ágiles, asume que las estimaciones son opcionales y en algunas ocasiones pueden convertirse en un desperdicio dado que tienen una alta probabilidad de no coincidir con la realidad.

#### **Estimaciones en proyectos pequeños**

Para estimar proyectos pequeños lo mejor es utilizar datos históricos propios de la organización. Las estimaciones en proyectos pequeños dependen en gran medida de las capacidades de los individuos que hacen el trabajo.

#### **Diferencias con las estimaciones en ambientes tradicionales**

##### **1. Respecto a las tres dimensiones**

En Agile se tiene fijo el tiempo (lo que dura la iteración) y los recursos afectados para trabajar, dejando variable el alcance. En los ambientes tradicionales, el alcance es lo que está fijo y el producto se encuentra definido.

##### **2. Respecto a la precisión**

Las estimaciones en los ambientes tradicionales buscan ser más precisas y la precisión en los ambientes ágiles y en correspondencia con la filosofía Lean, es considerada un desperdicio.

##### **3. Principal diferencia**

Las estimaciones ágiles son relativas, a diferencia de las estimaciones en ambientes tradicionales que son absolutas.

##### **4. Quien estima**

En las metodologías de estimación tradicional, las estimaciones las realiza el líder del proyecto o en ocasiones las realiza un experto. En los ambientes ágiles es el equipo el que realiza la estimación.

##### **5. Confusión con la planificación**

En los ambientes tradicionales existe un gran error que es asumir las estimaciones como compromisos, como si fueran planificaciones. Mientras que en agile, las estimaciones no implican un compromiso.

##### **6. Momentos para estimar**

En el enfoque tradicional se estima al comienzo del proyecto

En ágiles se estima al comienzo de cada sprint.



## SCRUM

Scrum es un marco ligero que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejos.

Scrum se basa en la inteligencia colectiva de las personas que lo utilizan. En lugar de proporcionar a las personas instrucciones detalladas, las reglas de Scrum guían sus relaciones e interacciones. Scrum se basa en el empirismo y el pensamiento Lean.

Scrum emplea un enfoque iterativo e incremental para optimizar la previsibilidad y controlar el riesgo.

Scrum involucra a grupos de personas que colectivamente tienen todas las habilidades y experiencia para hacer el trabajo y compartir o adquirir tales habilidades según sea necesario.

Scrum combina cuatro eventos formales para la inspección y adaptación dentro de un evento contenedor, el Sprint.

Estos eventos funcionan porque implementan los pilares empíricos de Scrum:

transparencia, inspección y adaptación.

### Pilares de Scrum

**Transparencia:** El proceso y el trabajo emergentes deben ser visibles para aquellos que realizan el trabajo, así como para los que reciben el trabajo.

**Inspección:** Los artefactos de Scrum y el progreso hacia objetivos acordados deben ser inspeccionados con frecuencia para detectar problemas potencialmente indeseables.

**Adaptación:** Si el proceso que se está aplicando o los materiales que se producen deben ajustarse. El ajuste debe realizarse lo antes posible para minimizar la desviación adicional.

### Valores de Scrum

El uso exitoso de Scrum depende de que las personas sean más competentes en vivir cinco valores:

Compromiso, Enfoque, Apertura, Respeto y Coraje

Estos valores dan dirección al equipo de Scrum con respecto a su trabajo, acciones y comportamiento.

### El equipo Scrum (Scrum Team)

La unidad fundamental de Scrum es un pequeño equipo de personas, un equipo Scrum. El equipo Scrum consta de un Scrum Master, un propietario de producto (Product Owner) y desarrolladores. Es una unidad cohesionada de profesionales enfocada en un objetivo a la vez, el objetivo del Producto.

En Los equipos de Scrum los miembros tienen todas las habilidades necesarias e internamente deciden quién hace qué, cuándo y cómo.

El equipo de Scrum es lo suficientemente pequeño como para permanecer ágil y lo suficientemente grande como para completar un trabajo significativo

Todo el equipo de Scrum es responsable de crear un incremento valioso y útil en cada Sprint.

### Desarrolladores

Los desarrolladores son las personas del equipo Scrum que se comprometen a crear cualquier aspecto de un

Incremento útil (funcional) en cada Sprint.

### Propietario del producto (Product Owner)

El Propietario del Producto es responsable de maximizar el valor del producto resultante del trabajo del equipo de Scrum. El Propietario del Producto también es responsable de la gestión eficaz de la pila del producto (Product Backlog)

Para que los Propietarios de Productos tengan éxito, toda la organización debe respetar sus decisiones.

El Propietario del Producto es una persona, no un comité.

### Scrum Master

El Scrum Master es responsable de establecer Scrum tal como se define en la Guía de Scrum. Lo consigue ayudando a todos a comprender la teoría y la práctica de Scrum. Lo logra al permitir que el equipo Scrum mejore

sus prácticas, dentro del marco de Scrum. Los Scrum Masters son verdaderos líderes que sirven al equipo Scrum y a toda la organización.

## Eventos de Scrum

El Sprint es un contenedor para todos los eventos. Cada evento en Scrum es una oportunidad formal para inspeccionar y adaptar los artefactos de Scrum.

Los eventos se utilizan en Scrum para crear regularidad y minimizar la necesidad de reuniones no definidas en Scrum. De manera óptima, todos los eventos se llevan a cabo al mismo tiempo y lugar para reducir la complejidad.

### El Sprint

Los sprints son el latido del corazón de Scrum, donde las ideas se convierten en valor.

Son eventos de longitud fija de un mes o menos para crear consistencia. Un nuevo Sprint comienza inmediatamente después de la conclusión del Sprint anterior. Todo el trabajo necesario para alcanzar el objetivo del producto

Los Sprints permiten la previsibilidad al garantizar la inspección y adaptación del progreso hacia un objetivo del Producto. Cada Sprint puede considerarse un proyecto corto.

Un Sprint podría ser cancelado si el Objetivo del Sprint se vuelve obsoleto. Solo el Propietario del Producto tiene la autoridad para cancelar el Sprint.

#### 1. Planificación de Sprint (Sprint Planning)

El Sprint Planning inicia el Sprint estableciendo el trabajo que se realizará para el mismo. Este plan resultante es creado por el trabajo colaborativo de todo el equipo de Scrum.

La planificación del Sprint aborda los siguientes temas:

Tema Uno: ¿Por qué este Sprint es valioso?

Tema dos: ¿Qué se puede hacer este Sprint?

Tema Tres: ¿Cómo se realizará el trabajo elegido?

#### 2. Scrum Diario (Daily Scrum)

El propósito del Daily Scrum es inspeccionar el progreso hacia el Objetivo Sprint y adaptar el Sprint Backlog según sea necesario. El Daily Scrum es un evento de 15 minutos (máximo) para los desarrolladores del equipo de Scrum.

Los Scrums diarios (Daily Scrum) mejoran la comunicación, identifican impedimentos, promueven una rápida para la toma de decisiones, y en consecuencia, eliminan la necesidad de otras reuniones.

#### 3. Revisión del Sprint (Sprint Review)

El propósito de la revisión del Sprint es inspeccionar el resultado del Sprint y determinar futuras

adaptaciones. El equipo de Scrum presenta los resultados de su trabajo a las partes interesadas clave

Durante el evento, el equipo de Scrum y las partes interesadas revisan lo que se logró en el Sprint. En base a esta información, se ve qué hacer a continuación.

#### 4. La retrospectiva del Sprint (Sprint Retrospective)

El propósito de la retrospectiva Sprint es planificar formas de aumentar la calidad y la eficacia.

El equipo de Scrum inspecciona cómo fue el último Sprint. El equipo de Scrum analiza qué fue bien durante el Sprint, qué problemas encontró y cómo esos problemas fueron (o no fueron) resueltos.

El equipo de Scrum identifica los cambios más útiles para mejorar su eficacia.

#### 5. Refinamiento del Product Backlog

### Artefactos de Scrum

Los artefactos de Scrum representan trabajo o valor.

#### 1. Pila del producto (Product Backlog)

El trabajo pendiente del producto es una lista emergente y ordenada de lo que se necesita para mejorar el producto. Es la única fuente de trabajo emprendida por el equipo Scrum.

*Compromiso: Objetivo del producto (Product Goal)*

El objetivo del producto (Product Goal) describe un estado futuro del producto que puede servir como objetivo para el equipo Scrum contra el cual planificar.

#### 2. La pila del Sprint (Sprint Backlog)

El Trabajo pendiente de Sprint es un plan por y para los desarrolladores. Es una imagen muy visible y en tiempo

real del trabajo que los desarrolladores planean realizar durante el Sprint para lograr el Objetivo Sprint. Por lo tanto, el Sprint Backlog se actualiza a lo largo del Sprint a medida que se aprende más.

*Compromiso: Sprint Goal*

El Sprint Goal es el único objetivo para el Sprint.

El objetivo de Sprint se crea durante el evento Sprint Planning y, a continuación, se agrega al Trabajo pendiente de Sprint. A medida que los desarrolladores trabajan durante el Sprint, tienen en cuenta el objetivo de Sprint.



3. Incremento (Increment)

Un Incremento es un paso de hormigón hacia el Objetivo del Producto. Cada Incremento es aditivo a todos los Incrementos anteriores asegurando que todos los Incrementos funcionen juntos.

*Compromiso: Definición de Hecho (Definition of Done)*

La Definición de Hecho es una descripción formal del estado del incremento cuando cumple con las medidas de calidad requeridas para el producto.

En el momento en que un elemento de trabajo pendiente de producto cumple con la definición de hecho, se crea un incremento.

**Timebox**

Esto existe para que no se pierda más tiempo del necesario en las distintas tareas y para aprender a negociar en términos del alcance del producto y no del tiempo o los costos

Si no se llega al final de un sprint, en vez de extender el tiempo, se entrega menos.

**Definición de Listo (DoR)**

El DoR es un criterio que define cuando una US está lo suficientemente bien formulada como para poderse incluir en un Sprint, y ser mostrada al PO, entonces él será el encargado de aprobar o no la US y en caso de que esté aprobada decidirá si desea ponerla en producción.

En el DoD se arma un checklist con todo lo que debe cumplir una US para entrar en producción.

**Capacidad del equipo en un Sprint**

La capacidad es una métrica que utiliza SCRUM para determinar cuánta cantidad de trabajo puede comprometer un equipo para un determinado Sprint.

**Niveles de Planificación**

| Nivel     | Horizonte                            | Quién                               | Foco                                                                                                                        | Entregable                                    |
|-----------|--------------------------------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Portfolio | 1 año o más                          | Stakeholders y Product Owners       | Administración de un Portfolio de Producto                                                                                  | Backlog de Portfolio                          |
| Producto  | Arriba de varios meses o más         | Product Owner y Stakeholder         | Visión y evolución del producto a través del tiempo                                                                         | Visión de Producto, roadmap y características |
| Release   | 3 (o menos) a 9 meses                | Equipo Scrum entero y Stakeholders  | Balancear continuamente el valor del cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto | Plan de reléase                               |
| Iteración | Cada iteración (de 1 semana a 1 mes) | Equipo Scrum entero                 | Que aspectos entregar en el siguiente Sprint                                                                                | Objetivo del Sprint y Sprint Backlog          |
| Día       | Diaria                               | Scrum Master y Equipo de desarrollo | Cómo completar lo comprometido                                                                                              | Inspección del progreso                       |



No pienso estudiarlo

**Herramientas de SCRUM**

Tarjeta de tarea

Es una tarjeta que cuenta con 5 partes:

- 1.Código del Item Backlog: es un identificador del ítem en el Product Backlog.
2. Nombre de la actividad: define qué es lo que hay que hacer.
3. Iniciales del responsable de realizar la tarea.
- 4.Número de día del sprint.
5. Estimación del tiempo para finalizar la tarea.

Tablero

Tablero utilizado por el Equipo de Desarrollo en el cual se colocan las tarjetas de tareas a realizar durante el sprint.

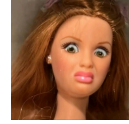
Se encuentra dividido en 5 secciones:

1. Story: el nombre de la historia.
2. To Do: aquí se encuentran las tareas por hacer en el Sprint.
3. Work In Process: aquí se encuentran las tareas que se están realizando.
4. To Verify: aquí se encuentran las tareas finalizadas que deben verificarse.
5. Done: aquí se encuentran las tareas terminadas: finalizadas y verificadas.

### Gráficos del Backlog

Gráficos que brindan información acerca del progreso de un Sprint, de una release o del producto.

*Después veré si tengo ganas de estudiar los tipos de gráficos*



## Gestión del Software como producto (SCM)

### Introducción

Los sistemas de software siempre cambian durante su desarrollo y uso. Conforme se hacen cambios al software, se crean nuevas versiones del sistema. Los sistemas pueden considerarse como un conjunto de versiones donde cada una de ellas debe mantenerse y gestionarse.

### Definición de Gestión de Configuración

Es una disciplina de soporte, que surge para enfrentar la crisis de problemas de software, monitoreando lo siguiente :

- Identificar características técnicas y funcionales de ítems de configuración y documentarlas;
- Controlar los cambios de esas características identificadas y documentadas, registrarlos y reportarlos;
- Verificar correspondencia con los requerimientos (trazabilidad).

Su propósito es resolver problemas y Algunos de esos problemas son:

- Pérdida de componentes;
- Pérdida de cambios (la versión del componente no es la última);
- Regresión de fallas;
- Doble mantenimiento;
- Superposición de cambios;
- Cambios no validados.

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.

La idea de la integridad del producto es hacer explícita las características o expectativas que tiene el cliente sobre el producto de software.

Mientras el producto existe hay actividad de gestión de configuración, que es responsabilidad de todo el equipo. Sin embargo, existen roles específicos como el rol del Gestor de configuración que tiene tareas adicionales como por ejemplo mantener la herramienta, marcar línea base, etc.

### ¿Cuál es el problema que se está tratando de solucionar?

El software es muy fácil de modificar, en el sentido de que uno puede entrar al repositorio y con un solo click eliminar meses de trabajo. Entonces lo que buscamos con la gestión de configuración del software es tratar de que estas cosas no pasen, buscando que el producto de software sea íntegro y si fuera el caso de que se van a realizar cambios, podamos tener un control de qué cosas cambiaron en un determinado momento. Es por esta razón que se asocia el concepto de software con el concepto de versión, ya que cada ítem de configuración debe tener su versión.

Y todas las personas se enteren de los cambios.

### CONCEPTOS GENERALES

#### Ítem de Configuración

Son aquellos artefactos que forman parte del producto o proyecto, y pueden ser almacenados en un repositorio. Pueden sufrir cambios, y se desea conocer su estado y evolución a lo largo del ciclo de vida. Es decir, es cualquier aspecto asociado al producto de software. Son todos aquellos elementos que componen toda la información producida como parte del proceso de ingeniería de software, como ser programas de computadora (código fuente y ejecutables), documentos que describen los programas (documentos técnicos o de usuario), datos (de programa o externos).

Algunos ejemplos: prototipo de interfaz, manual de usuario, ERS, arquitectura del software, casos de prueba, código fuente, íconos, etc, etc.

## **Versión**

Instancia de un ítem de configuración que difiere de otras instancias de este ítem. Controlar una versión refiere a la evolución de un único ítem de configuración. Cada versión de software es una colección de elementos de configuración (ECS)

La versión es un punto particular en el tiempo de ese ítem de configuración (es un estado).

Una versión se define como la forma particular de un artefacto en un instante o contexto dado.

El **control de versiones** se refiere a la evolución de un único ítem de configuración (IC)

La evolución puede representarse gráficamente en forma de grafo.

## **Variante**

Versión de un ítem de configuración (o de la configuración) que evoluciona por separado. Las variantes representan configuraciones alternativas. Si la diferencia que existe entre las instancias de un mismo ítem de configuración es muy pequeña, no se denomina versión, sino que se la conoce como variante. Está ligado a trabajar con Branches.

## **Configuración**

Conjunto de todos los ítems de configuración con su versión específica. Define una foto de los ítems de configuración en un momento de tiempo determinado.

La configuración de un software es la sumatoria de todos los ítems de configuración que tiene en un momento determinado, equivale a una instantánea o una foto de todos los ítems de configuración con su versión en un momento del tiempo.

### **Repositorio**

Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos. Tiene una estructura para mantener el orden y la integridad. Tenemos

2 tipos de repositorios:

1. **Repositorios Centralizados**: está caracterizado porque un servidor contiene todos los archivos con sus versiones.

Los administradores tienen un mayor control sobre el repositorio. Tiene como desventaja qué si falla el servidor, todo lo positivo se cae.

2. **Repositorios Descentralizados**: está caracterizado porque cada cliente tiene una copia exactamente igual del repositorio completo. Tiene como ventaja que se resuelve el problema de los servidores centralizados. La desventaja que podemos mencionar es que es más complicado llevar un control sobre el mismo.

### **Release**

Entrega de un sistema que se libera para su uso a los clientes u otros usuarios de la organización.

### **Rama (Branch)**

Es un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, por varios motivos:

- Experimentación;
- Resolución de errores en el desarrollo;
- Desarrollar un mismo software para diferentes plataformas.

### **Integración de ramas (Merge)**

Se da cuando se integran dos ramas, para fusionar la configuración de los ítems de configuración con sus correspondientes versiones. Una buena práctica es mantener la rama principal como la versión estable, y fusionar los cambios hacia ella.

### **El rol de las líneas base y su administración**

Es un conjunto de ítems de configuración que han sido construidos y revisados formalmente, de manera que pueden ser tomados como referencia para demostrar que se ha alcanzado cierto nivel de madurez en ellos, y que sirve como base para desarrollos posteriores.

Este conjunto de ítems debe tener una referencia única, y esto se hace a través de “tags”.

Se acuerdan parámetros para determinar qué se considera o qué debe cumplir los ítems de configuración para considerarse como línea base.

¿Para qué sirve una línea base?

Fundamentalmente es para tener un punto de referencia, pero también sirve para hacer Rollback o saber qué se pone en producción.

Existen dos tipos de línea base:

1. **Operacionales**: contiene una versión de producto cuyo código es ejecutable, Es la línea base de productos que han pasado por un control de calidad definido previamente.

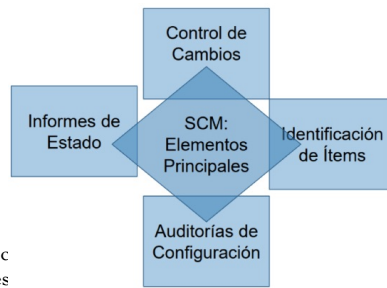
2. **De especificación o documentación**: son las primeras línea base, dado que no cuentan con código.

## Planificación de la gestión de configuración de software

Este plan se debe confeccionar al inicio de un proyecto

Consideraciones:

- Debe hacerse tempranamente;
- Se deben definir los documentos que se administrarán;
- Todos los productos del proceso deben administrarse.



## Actividades relacionadas a la gestión de configuración

Estas actividades/elementos son las cosas que contienen las secciones de un plan de gestión de configuración. Recordemos que la gestión de configuración también se debe planificar.

### Identificación de Ítems de Configuración

esto implica una identificación unívoca para cada ítem, donde en el equipo se definirán políticas y reglas de nombrado y versionado para todos ellos. También, se debe definir la estructura del repositorio, y la ubicación de los ítems de configuración dentro de esa estructura.

Además, implica una definición cuidadosa de los componentes de la línea base.

Esta identificación y documentación proveen un camino que une todas las etapas del ciclo de vida del software,

Se clasifican en:

1. *Ítem de producto*: tienen el ciclo de vida más largo, y se mantienen mientras el producto exista. Ejemplo: documento de arquitectura, casos de uso, código, manual de usuario y de parametrización, casos de prueba, una ERS, los casos de prueba, la base de datos.
2. *Ítems de proyecto*: el plan de proyecto, el listado de defectos encontrados
3. *Ítems de iteración*: Conocer el ciclo de vida de un ítem permite establecer la nomenclatura de este.

Luego de identificar los ítems se debe armar la estructura del repositorio con nombres representativos. Luego se vinculan los ítems de configuración con el lugar físico del repositorio donde se van a almacenar.

### Control de cambios

Una vez definida la línea base, no es posible cambiarla sin antes pasar por un proceso formal de control de cambios.

Es decir que el control se hace sobre los ítems de configuración que pertenecen a la línea base.

Este proceso es llevado a cabo por el *comité de control de cambios*.

*Etapas del proceso*

en caso de recibir una propuesta de cambio, los pasos son:

1. Se recibe una propuesta de cambio sobre una línea base determinada, no sobre un ítem.
2. El comité de control de cambios realiza un análisis de impacto del cambio
3. En caso de que se autorice la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar
4. Luego de realizado el cambio, el comité vuelve a intervenir para aprobar la modificación
5. Finalmente se notifica a los interesados los cambios realizados sobre la línea base.

### Auditorías de configuración

Son controles autorizados a realizar por el equipo de desarrollo en un momento determinado, donde un auditor externo al equipo (independiente y objetivo) analiza las líneas base, las cuales permiten “congelar” y analizar en un momento determinado cuál es el estado del software, y si se están cumpliendo todas las pautas que plantea esta disciplina de SCM.

En metodologías ágiles, esta es la única actividad de la disciplina SCM que no se soporta por la filosofía.

Las auditorías permiten visualizar si se están satisfaciendo los requerimientos

Tipos de auditorías

1. Auditoría física o de contingencia: se realiza primero. Se validan los ítems de configuración de la línea base contra el plan de gestión de configuración, asegurando que la documentación que se entregará es consistente con el código desarrollado. Si esto no pasa, la auditoría funcional no se realiza.

2. Auditoría funcional de configuración: audita la trazabilidad de requerimientos, controlando que la funcionalidad y performance real de cada ítem de configuración sean consistentes con la especificación de requerimientos.

#### Beneficios:

- Aumenta la protección contra cambios innecesarios;
- Mejora de la visibilidad de estado del proyecto y sus componentes;
- Aumenta la auto responsabilidad;
- Disminuye los costos por retrabajo;
- Disminuye el tiempo de desarrollo;
- Aumenta la calidad;
- Suministra viabilidad y rastreabilidad del ciclo de vida del producto de software.

#### Desventajas:

- Quejas (es burocrático, molesto, se meten con mi trabajo);
- La transición es difícil;
- No hay compromiso en todos los niveles;
- No hay consciencia del problema.

### Informe de estado

proporciona un mecanismo para mantener un registro de cómo evoluciona el sistema. Esto permite mantener a todo el equipo informado sobre la última línea base, y que se trabaje en base a su última versión, y no sobre una versión obsoleta.

Estos reportes incluyen todos los cambios que han sido realizados a las líneas base durante el ciclo de vida del software.

Permite responder a preguntas como:

- ¿Cuál es el estado del ítem?
- ¿La propuesta de cambio fue aceptada o rechazada por el comité?
- ¿Qué versión de ítem implementa un cambio de una propuesta de cambio aceptada?
- ¿Cuál es la diferencia entre dos versiones de un mismo ítem?

### **Gestión de configuración en ambientes ágiles**

En las metodologías ágiles la gestión de configuración cambia el enfoque, ya que la misma es de utilidad para los miembros del equipo de desarrollo y no viceversa.

la gestión de configuración posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores.

Los equipos ágiles son autoorganizados, por lo que las Auditorías de configuración no son una actividad propia de la gestión de configuración en los ambientes ágiles. No existe comité

La diferencia es que el manifiesto ágil se enfoca en los proyectos, mientras que la gestión de configuración de software se enfoca en el producto.

### Características

1. Dada la naturaleza de los requerimientos ágiles, la SCM responde a los cambios en lugar de evitarlos
2. La automatización debe ser aplicada donde sea posible.
3. Provee retroalimentación continua sobre calidad, estabilidad e integridad.
4. Las actividades de SCM se encuentran embebidas en las demás tareas para alcanzar el objetivo del sprint.
5. Es responsabilidad de todo el equipo, por lo tanto, requiere capacitación.
6. Adopción de las prácticas continuas.

### **Continuous Integration**

Es una práctica de desarrollo que promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día, permitiéndole al equipo detectar problemas de manera temprana.

### **Continuous Delivery**

Es una disciplina de desarrollo de software en la que el software se construye de tal manera que puede ser liberado en producción en cualquier momento. Se prioriza que la versión esté en un estado en el que pueda ser puesta en producción.

### **Continuous Deployment – Estrategias de Deployments – Canary Deployments – Blue/Green Deployment**

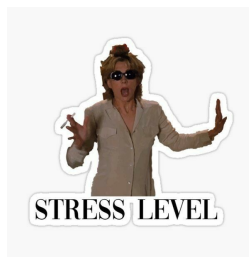
Consiste en poner en el ambiente de producción del usuario final el producto. A diferencia de la entrega continua, en el despliegue continuo no existe la intervención humana para desplegar el producto en producción. Para esto, se utilizan pipelines que contienen una serie de pasos que deben ejecutarse en un orden determinado para que la instalación sea satisfactoria.

### Canary Deployment

En ingeniería de software, Canary deployment es la práctica de realizar lanzamientos por etapas. Primero implementamos una actualización de software para una pequeña parte de los usuarios, para que puedan probarla y proporcionar comentarios. Una vez aceptado el cambio, la actualización se extiende al resto de usuarios.

### Blue-Green Deployment

Es un modelo de lanzamiento de aplicaciones que transfiere gradualmente el tráfico de usuarios de una versión anterior de una aplicación o microservicio a una nueva versión casi idéntica, las cuales se ejecutan en producción.



Temas que se evalúan en el primer parcial

**Unidad 1 completa** (incluye paper No silver bullet)

- Introducción a la Ingeniería del Software. ¿Qué es?
- Estado Actual y Antecedentes. La Crisis del Software.
- Disciplinas que conforman la Ingeniería de Software.
- Ejemplos de grandes proyectos de software fallidos y exitosos.
- Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.
- Procesos de Desarrollo Empíricos vs. Definidos.
- Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software
- Ventajas y desventajas de c/u de los ciclos de vida.
- Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.
- Componentes de un Proyecto de Sistemas de Información.
- Vínculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.

Unidad 2

- Gestión de Producto
- Requerimientos Ágiles
- User Stories
- Estimaciones Ágiles
- Framework Scrum

Unidad 3

- SCM

Y aún  
así me  
quedé